RSF 1.0: Recursive Semantic Framework

A Six-Slot Semantic Framework for Agent Communication

Defining the Recursive Semantic Framework — A Minimal, Traceable, and Auditable Semantic Representation Protocol for Intelligent Systems

Author: Wang Daqing

Version: 1.0 **Release Date**: July 2026 **License**: CC BY 4.0

Keywords: RSF, Recursive Semantic Framework, Semantic Representation, Six-Slot Structure, Agent Communication, Semantic Drift, Traceability, Auditability

---

Abstract

RSF (Recursive Semantic Framework) is a structured semantic intermediate representation protocol for intelligent systems. Its core is a fixed six-slot skeleton — **A (Agent)**, **B (Action)**, **C (Target)**, **D (Time)**, **E (Environment)**, **F (Context)** — designed to extract event semantics from natural language and enable direct semantic exchange between intelligent systems, rather than relying on natural language as an intermediary.

RSF is not a new language, not a replacement for traditional grammar systems, and not a competitor to LLMs or Transformers. It is a **semantic interface layer** between natural language and intelligent systems.

RSF's value is built around three core properties: **Consistency** (all Agents work from the same semantic structure, eliminating discrepancies from individual NL parsing), **Traceability** (every output fragment can be mapped back to a specific slot in the original input, making the semantic path reviewable), and **Auditability** (structured six-slot logs support item-by-item inspection of decision rationale, suitable for high-reliability domains such as finance, law, and healthcare).

This document records the core ideas, structural definitions, design principles, and application examples of RSF, including multi-agent chain communication, high-fidelity domain translation, and structured RAG scenarios. RSF is currently at the conceptual

framework and preliminary case validation stage, with no automated parser or large-scale system deployment. The purpose of this document is to establish a timestamped record of original ideas and provide a public reference for subsequent research.

---

1. Introduction: Communication Is About Meaning, Not Language

1.1 The Convenience Store: Communication Without Language

A foreign tourist walks into a convenience store. The tourist does not speak the local language, and the cashier does not speak the tourist's language. They share no common language. The tourist wants to buy two bottles of cola — without saying a word, he walks to the cooler, takes out two bottles, and places them on the counter. The cashier observes this, then raises two fingers. The tourist understands this gesture as representing a quantity or price relationship, pays two dollars. The cashier accepts the payment. The transaction is complete.

No language was exchanged, no text was written, no translation occurred — yet communication succeeded.

Why? Because both parties shared a constraining semantic environment. The convenience store itself provided crucial semantic constraints: picking up a bottle of cola could mean stocking shelves in a warehouse, drinking at a friend's house, or — in a convenience store — an intention to purchase. The environment limited the interpretation space. Similarly, the cashier raising two fingers could mean the number 2, second place, a victory sign, or something else entirely — but in the transaction context, its meaning was constrained by the environment and shared knowledge.

Thus, communication does not depend on one-to-one correspondence between symbols. It depends on: **symbols + environmental constraints + shared knowledge + semantic understanding**.

1.2 Language and Meaning Belong to Different Layers

The convenience store case reveals an important fact: the core of communication is not language itself, but the meaning structure behind language. Language is just one representation of semantics, not semantics itself.

This distinction seems simple, but it points to a long-overlooked question: when intelligent systems exchange information, must they rely on natural language the way humans do? Or can they exchange meaning directly?

Different languages can express the same underlying semantics:

```
English: The company completed the project.
Chinese: 公司完成了项目。
```

The two sentences have different surface forms, but they describe the same event: a subject performed an action on an object. Languages are diverse, but semantic structures have a higher-order consistency.

1.3 Semantic Drift: The Hidden Cost in Multi-Agent Systems

Current multi-agent communication still relies heavily on natural language. The typical process is:

```
Agent A → generate natural language → Agent B → re-parse
natural language → extract meaning → continue processing
```

This approach directly inherits human communication patterns. It is flexible, but it has a fundamental problem: every communication requires re-extracting meaning from linguistic form.

More critically, there is a known but under-addressed risk — **Semantic Drift**. When Agent A converts its internal semantics into natural language, the first information loss occurs (encoding loss); when Agent B reconstructs semantics from natural language, the second loss occurs (decoding loss). In long-chain multi-agent collaboration, each NL generation → NL parsing cycle introduces an uncontrolled semantic shift.

Consider a scenario: Agent A (customer service) tells Agent B (warehouse) "the customer returned a defective laptop." Agent B, after processing, tells Agent C (finance) "process a refund for this returned item." By the time it reaches Agent C, "defective" may have degraded to simply "returned," the refund conditions may be lost, and whether the refund includes shipping costs may not be mentioned at all. Information undergoes untraceable changes with each Agent hop.

This is not a problem of natural language being "insufficiently expressive" — natural language is extremely powerful for expressing complex semantics. The problem is that

natural language is **optimized for human communication**, not for precise, efficient, and verifiable information exchange between machines.

1.4 The Motivation for RSF

Based on the above observations, RSF (Recursive Semantic Framework) attempts to answer one question: **how can intelligent systems represent and exchange meaning more directly**, rather than relying on natural language as an intermediary every time?

RSF does not seek to replace natural language — natural language will remain an important medium for human-machine interaction. What RSF focuses on is whether, within intelligent systems, information exchange can be based on semantic structures rather than linguistic forms.

The following chapters define what RSF is, how it works, and its preliminary applications in multi-agent communication and high-fidelity domain translation.

---

2. RSF: Definition and Core Structure

2.1 What Is RSF

RSF (Recursive Semantic Framework) is a structured semantic intermediate representation protocol for intelligent systems. Its core objective is to extract event semantics from natural language expressions and represent them using a fixed six-slot structure, enabling different intelligent systems to exchange information directly at the semantic level without needing to re-parse natural language each time.

RSF is not:

- A new natural or artificial language
- A replacement for traditional grammar systems
- A competitor to LLMs or Transformers
- A communication protocol (such as MCP or A2A)

RSF is: a *semantic interface layer* between natural language and intelligent systems.

2.2 The Six-Slot Definition

RSF represents a basic semantic event as six slots:

```
Event = A(Agent) + B(Action) + C(Target) + D(Time) +
E(Environment) + F(Context)
```

Slot	Name	Description
A	Agent	The entity performing the action
B	Action	The verb and its complete inflections
C	Target	The object of the action
D	Time	Time point, duration, etc.
E	Environment	Space, location, etc.
F	Context	Manner, reason, etc. — unified into F

Example:

```
The company completed the project yesterday in Tokyo.

A          B            C          D           E          F
Company    completed    project    yesterday   Tokyo      Ø
```

RSF is not concerned with "what words make up the sentence," but rather "what event structure this expression describes." The same event, whether expressed in English, Chinese, or Japanese, should yield the same six-slot skeleton after RSF extraction.

2.3 Recursion: Six Slots Within Six Slots, No New Slot Names

The word "Recursive" in RSF's name represents a core design idea: complexity comes from recursive expansion of the structure, not from an increase in the number of rules.

Recursion is defined as: when any of the six slots contains a complete event inside it, the same A–F framework can be applied one level deeper, without introducing new slot names.

Example:

```
The researchers who conducted the study analyzed the data.

First level:
A                                                B
C
The researchers [who conducted the study]        analyzed
```

```
the data

Recursive entry into slot A (relative clause):
A                  B           C
who (researchers) conducted   the study
```

The recursive level still uses A, B, C — not A1, B1, C1. Whether the nesting depth is one level or ten, the parsing rules remain identical. The Core does not require any additional rules for deep nesting.

2.4 Output Modes

Mode	Content	Use Case
Lite Mode	Final result only (translation or response)	Daily use
Standard Mode	Result + symbol markup	Basic traceability needed
Audit Mode	Result + tree JSON + adjustment records	High-reliability scenarios

2.5 Relationship to Existing Communication Protocols

MCP (Model Context Protocol) and A2A (Agent-to-Agent) address how Agents connect, discover tools, and transmit data. RSF is not in competition with them:

Layer	Responsibility	Example
Communication Protocol Layer	How Agents connect and how data is transmitted	API / MCP / A2A
Semantic Representation Layer (RSF)	What meaning the Agents exchange	Six-slot structured semantics
Application Layer	Specific business logic	Financial transfer / Contract review

MCP helps Agents discover tools, A2A helps Agents establish communication, and RSF helps Agents express tasks, states, conditions, goals, and decision rationales more clearly. The three can coexist.

2.6 Three Core Properties

RSF's value for intelligent systems is built around three core properties.

Consistency: All participating Agents work from the same semantic structure. When Agent-A extracts information into a six-slot semantic package, Agent-B and Agent-C read exactly the same slot structure — they do not need to each re-parse the semantics from natural language. This means the same event will not be interpreted differently due to linguistic differences between Agents. Consistency also extends to cross-language scenarios — the same event in English and Chinese yields an identical six-slot skeleton, eliminating the problem of "semantic distortion when switching languages."

Traceability: Every output fragment can be mapped back to a specific linguistic fragment in the original input and located to a corresponding A–F slot. Agents can trace "which sentence in the original input this information comes from, and which slot it belongs to." The six-slot structure itself is a natural indexing system: when the system needs to answer "where this amount came from," it can backtrack along the Agent chain to the specific slot and timestamp where the amount was originally extracted. Traceability turns the semantic path from a black box into a reviewable chain.

Auditability: Building on traceability, RSF's Audit mode provides complete structured logs — each Agent's input and output is a six-slot JSON, and every step's slot values, adjustment records, and decision rationale can be inspected item by item. For high-reliability domains such as finance, law, and healthcare, this means the system can answer "why this conclusion was reached" — not by inspecting the uninterpretable internal state of a model, but by inspecting a structured semantic path.

These three properties support each other: consistency ensures all Agents work on the same semantic baseline, traceability ensures every decision path can be backtracked, and auditability ensures the entire reasoning chain can be externally verified. Together, they constitute RSF's core differentiation from both pure natural language communication and ordinary JSON Schema.

---

3. Design Principles

3.1 Fixed Skeleton

RSF's most fundamental design decision is: the top-level slots are fixed at six and cannot be increased.

The starting point for this decision is not "six dimensions are sufficient to describe all semantics" — no finite structure can achieve that. The starting point is **Closure**: within the boundary of six slots, any event-type semantic expression can find a home. Information beyond the six-slot boundary is left to the AI's own intelligence.

The value of a fixed skeleton is: the AI system does not need to re-judge "what is the structure of this sentence" each time it reasons — the answer is always those six slots.

RSF assumes that the underlying AI system already possesses basic natural language understanding capabilities — it can identify slot boundaries and perform recursive parsing. This assumption is verifiable on current mainstream LLMs.

3.2 Packaging First

Traditional language analysis tends toward splitting — breaking verbs into tense, modality, voice, etc., and splitting fixed collocations into individual words. The finer the split, the more complex the relationships between semantic fragments become, and the higher the cost of recomposition.

RSF does the opposite: **package as a whole**.

B-slot integration: The complete semantics of an action — the verb itself along with its tense, modality, fixed collocations, etc. — are placed into the B-slot as a whole. For example, "should have been completed" goes into B as one unit, rather than being split into four independent components.

F-slot unification: All modifying semantics — manner, instrument, reason, etc. — are unified into the F-slot. The Core does not subdivide them.

The direct benefit of Packaging First is: the AI does not need to recompose the grammatical relationships between these components during reasoning. Tense, modality, etc., are already locked inside the B-slot; the AI only needs to understand them as a whole.

3.3 Traceability First

RSF's core priority is: **fidelity and traceability take precedence over readability**. Every output fragment must be mappable back to an A–F slot. Key information (facts, modality, negation, quantity, person, causal intensity) must be fully preserved. When readability conflicts with fidelity, RSF sacrifices some readability rather than allowing information loss or slot ambiguity.

Only after fidelity and traceability are guaranteed should output naturalness be optimized.

3.4 Flatness and Recursion

RSF simultaneously upholds two seemingly contradictory design properties: structural flatness and recursive depth. In reality, they do not conflict.

Flatness means: the top level consists of only six slots, with no sub-slots and no mandatory Topic/Focus layer. This is a width constraint.

Recursion means: when any slot contains a complete event, the same A–F framework can be applied one level deeper, without introducing new slot names. This is a depth capability.

Recursion does not violate flatness, because recursion does not add new slot types — no matter how many levels deep, the slots used are still A, B, C, D, E, F.

3.5 Round-Trip Conservation

RSF defines a rigorous verification standard — **Round-Trip Conservation**: converting text from the source language into a six-slot structure, generating the target language from the six-slot structure, and then translating the target language back into the source language — the result must match the original input on all key information. Modality, tense-aspect, negation, quantity, person, and causal intensity — deviation on any single dimension means fidelity has been compromised.

3.6 Core Closure

The Core pursues **Closure**, not Completeness.

---

4. Application Examples

4.1 Simple Event: Convenience Store Purchase

Starting with the convenience store story from Chapter 1. The event of a tourist buying two bottles of cola is represented in RSF as:

A	B	C	D	E	F

Customer	purchase	two bottles of cola	∅	convenience store	payment=$2; condition=no common language

The convenience store story in Chapter 1 already contained the complete six-slot semantics — they were simply not annotated at the time. What RSF does is make these implicit semantic roles explicit.

4.2 Translation Fidelity Example

```
The report should have been completed by the team last
week.
```

A	B	C	D	E	F
the team	should have been completed	the report	last week	∅	∅

Traditional grammar would split "should have been completed" into a modal verb (should) + perfect aspect (have been) + passive voice (completed). RSF packages the entire phrase into the B-slot, because these three components jointly express an indivisible semantic unit: "something that ought to have been finished in the past, but it is uncertain whether it was actually finished."

Round-trip conservation check: from the B-slot "should have been completed," generate the Chinese translation "应该已经被完成," then translate it back to English — tense, modality, and passive voice are all preserved without loss or attenuation.

4.3 Multi-Agent Chain Communication: Return Processing

This is the most important case study, demonstrating how RSF transmits semantics across multiple Agents while blocking semantic drift.

Scenario

Three Agents in an e-commerce system collaborate to process a customer return:

Agent	Role
Agent-A	Customer service Agent, receives the customer's request
Agent-B	Warehouse Agent, handles return receipt and quality inspection

Agent-C	Finance Agent, executes the refund

Step 1: Customer Input

```
"我要退掉上周买的那台有缺陷的 BrandX Pro-15 笔记本电脑，订单号是
ORD-20260703-8821。"
(I want to return the defective BrandX Pro-15 laptop I
bought last week, order number is ORD-20260703-8821.)
```

Step 2: Agent-A Extracts RSF Semantic Package

```
{
  "A": "Customer (CUST-48291, Zhang San)",
  "B": "return + request_inspection + conditional_refund",
  "C": {
    "item": "BrandX Pro-15 Laptop",
    "order_id": "ORD-20260703-8821"
  },
  "D": {
    "purchase_date": "2026-07-03",
    "return_initiated": "2026-07-10"
  },
  "E": "∅",
  "F": {
    "defect_description": "intermittent_black_screen",
    "return_label": "issued_by_Agent-A_via_email",
    "condition_for_refund": "confirm_defect_first",
    "unresolved_question":
["does_refund_include_original_shipping"]
  }
}
```

Note the `unresolved_question` field in the F-slot: Agent-A proactively marks information it is uncertain about (whether the refund includes original shipping), rather than pretending the information is complete.

Step 3: Agent-A → Agent-B (Transmit Semantic Package)

Agent-A transmits the complete RSF semantic package to Agent-B without generating natural language.

Step 4: Agent-B Reads and Processes

Agent-B directly reads the six-slot structure:

- **A** = Customer CUST-48291 → can look up customer history
- **B** = return + inspection + conditional_refund → task chain is clear
- **C** = BrandX Pro-15, order ORD-20260703-8821 → precise SKU and order number
- **F.defect_description** = intermittent_black_screen → specific inspection criteria
- **F.unresolved_question** = does_refund_include_original_shipping → Agent-B knows this issue is unresolved

After performing the quality inspection, Agent-B updates the semantic package and passes it to Agent-C:

```
{
  "A": "Agent-B (Warehouse-R2D2)",
  "B": "confirm_defect + forward_for_refund",
  "C": {
    "order_id": "ORD-20260703-8821",
    "customer": "CUST-48291",
    "item": "BrandX Pro-15"
  },
  "D": "2026-07-11",
  "E": "warehouse-R2D2",
  "F": {
    "inspection_result":
"confirmed_intermittent_black_screen",
    "refund_authorization": {
      "approved": true,
      "amount": 8999.00,
      "currency": "CNY",
      "includes_shipping": true,
      "policy_reference": "refund_policy_v4.2_section_3.1"
    },
    "resolved_questions": {
      "does_refund_include_original_shipping":
"yes_per_policy_v4.2"
    },
    "action_required_by": "Agent-C (Finance)"
  }
}
```

Step 5: Agent-C Executes Directly

Agent-C does not need to ask any follow-up questions. It executes the refund directly.

Comparison: NL Communication vs RSF Semantic Package

Dimension	NL Communication	RSF Semantic Package
Information precision	"a defective laptop" — vague	brand/model/order_id/defect all in slots, zero ambiguity
Clarification rounds	At least 1–2 rounds of follow-up questions	Zero additional rounds
Unresolved issues	Agent-A may forget to mention "does refund include shipping"	F.unresolved_question proactively marks it; the issue is never lost
Semantic drift	Information degrades through two NL encoding/decoding cycles	Direct transmission, zero drift
Audit	Relies on NLP to understand text logs	Slot-by-slot inspection of structured JSON

Audit Trail

```
Input (Customer NL)
  ↓
Agent-A: RSF Extraction
  └─ A: Customer CUST-48291
  └─ B: return + inspection + conditional_refund
  └─ C: BrandX Pro-15 / ORD-20260703-8821
  └─ F: unresolved_question [shipping_fee]
  ↓
Agent-B: Inspection + Update
  └─ B: confirm_defect → forward_for_refund
  └─ F: resolved_questions [shipping_fee: yes]
  └─ F: refund_authorization [amount: 8999.00]
  ↓
Agent-C: Execute Refund
  └─ Action: refund_approved
  └─ Amount: 8999.00 CNY
  └─ Policy: refund_policy_v4.2_section_3.1
```

4.4 High-Fidelity Domain Translation Examples

RSF's six-slot structure and Packaging First principle are naturally suited to domains that demand high information fidelity — such as law, finance, and healthcare. The

following two examples demonstrate how RSF preserves critical semantics in these domains.

Legal Clause

```
"The Lessee shall, within 30 days after receipt of the
notice, remedy the breach,
failing which the Lessor may terminate this Agreement."
```

A	B	C	D	F
Lessee	shall remedy	the breach	within 30 days after receipt of notice	condition=failing which → Lessor may terminate

Fidelity point: The B-slot's "shall remedy" preserves the obligatory force of "shall" in legal English — it is neither "may" (permissive) nor "must" (more imperative), but the standard modal for legal obligations. The F-slot captures the "failing which" conditional relationship, which is the critical logical connector in this clause — if the Lessee fails to remedy within 30 days, the Lessor's termination right is triggered. Traditional translation might simplify "failing which" to "otherwise," losing the precise "condition → consequence" logical relationship of legal text.

Chinese translation under RSF: "承租人应在收到通知后30天内补救该违约行为，否则出租人可终止本协议。"

Medical Record

```
"No evidence of metastasis was found in any of the 12 lymph
nodes examined."
```

A	B	C	F
∅	was found	no evidence of metastasis	scope=all 12 lymph nodes examined; negation=full

Fidelity point: This is a typical medical report sentence where negation scope and quantification are critical. The C-slot "no evidence of metastasis" and the F-slot's "scope=all 12 lymph nodes examined" together preserve the negation scope — it is not "metastasis was not found in some of the lymph nodes," but "all 12 lymph nodes were examined and none showed evidence of metastasis." If the negation scope were

misinterpreted (e.g., understood as "12 lymph nodes, some of which showed no evidence"), it would directly affect clinical diagnosis. The empty A-slot also faithfully reflects the medical writing convention of not specifying the agent.

Chinese translation under RSF: "所检查的12枚淋巴结中均未发现转移证据。"

4.5 Structured RAG Example

In RAG scenarios, a user query can first be extracted into a six-slot structure, then used for retrieval. For example, the query "去年第三季度的营收数据" (revenue data for Q3 last year):

A	B	C	D	E	F
∅	query	revenue data	Q3 2025	∅	data_source=financial_reports

The retrieval system can use a combination of the D-slot (Q3 2025) and C-slot (revenue data) for precise retrieval, rather than performing vector matching on the entire NL query. This can improve retrieval accuracy in structured query scenarios.

---

5. Boundaries and Positioning

5.1 Distinctions from Related Work

AMR (Abstract Meaning Representation): AMR represents sentences as directed graphs, with nodes as semantic concepts and edges as relations. AMR pursues linguistic completeness, but at the cost of high graph complexity and parsing cost. RSF pursues a minimal executable event skeleton, not a complete semantic graph.

SRL (Semantic Role Labeling): SRL identifies predicate-argument structures in sentences, with role sets varying by predicate type. RSF uses fixed six slots. Fixed means lower parsing complexity, but also coarser granularity — this is a trade-off made for engineering executability.

FrameNet: FrameNet is based on frame semantics, where each word activates a frame. FrameNet is extremely rich, but its coverage is limited to annotated frames. RSF does not adopt a frame-based system.

CNL (Controlled Natural Language): CNL eliminates ambiguity by restricting grammar and vocabulary. RSF is not CNL — it does not restrict how users input

language. It accepts full natural language input and extracts semantic structure at the understanding layer.

MCP / A2A: Communication protocols handle "how to transmit"; RSF handles "what to transmit." They can coexist.

5.2 RSF's Positioning

RSF prioritizes **"minimal executable event skeleton + engineering auditability"** over linguistic completeness.

---

6. Current Status, Limitations, and Future Directions

6.1 Current Status

RSF is currently at the conceptual framework and preliminary case validation stage. The core specification (constitution) has been frozen, but there is no automated RSF parser or large-scale system deployment. All examples in this document are manually annotated or extracted with the assistance of LLMs.

6.2 Known Limitations

No automated parser yet: Current six-slot extraction is done manually or with LLM assistance. There is no dedicated RSF parsing engine. Automating the conversion from natural language to six-slot structure is the key bottleneck for RSF implementation.

Coverage not systematically validated: Can six slots cover all expression types in all languages? Preliminary manual validation exists, but there has been no large-scale testing. Coverage of edge cases (poetry, metaphor, irony) is unknown.

Agent communication not deployed in real systems: The Agent example in Chapter 4 is a conceptual demonstration. It has not been implemented and tested in a real multi-agent system. Quantitative metrics such as token savings and reduction in communication rounds have yet to be measured.

Integration cost unknown: How much re-engineering is required to migrate an existing Agent system from NL communication to RSF semantic package communication? There is no practical experience yet.

6.3 Future Directions

Benchmark design: A reproducible test framework is needed to evaluate RSF's practical benefits. Controlled experiments should compare NL communication and RSF semantic package communication on the same tasks, measuring token consumption, communication rounds, consistency, semantic drift magnitude, and audit pass rate.

Open-source parser: Develop an automated parser from natural language to six-slot structures — whether rule-based, fine-tuned LLM, or a hybrid approach.

Cross-language validation: Can significantly different languages — Chinese, English, Japanese, Arabic — all be adequately covered by six slots? Where are the boundaries?

Encoder-side preprocessing: Adding an RSF preprocessing step at the front end of Transformer models (such as LLMs or T5) to distill long texts into six-slot skeletons before input — this direction is currently at the concept stage and requires concrete engineering experiments to verify feasibility.

Declaration

Specification: RSF (Constitution Edition)

Status: Core Frozen

Author: Wang Daqing **Version**: 1.0 **Release Date**: July 2026